SCIENCE CHINA
Information Sciences

• RESEARCH PAPER •

Population-size fidelity in reference-vector multi-objective evolutionary algorithms

Yuxuan ZHANG[1*]

[1]*Graduate School, Army Engineering University of PLA, Nanjing 210007, China*

Abstract A computational experiment is only as credible as the quantities it claims to control. The population size of a reference-vector multi-objective evolutionary algorithm can silently stop being one of them; we turn this into an audit framework, with PlatEMO 4.16 as the case study. The framework separates three quantities — requested, realised, reported — and provides two propositions, an audit, a repair catalogue and a reporting protocol. It finds the assignment in at least 84 of 1887 source files, covering at least 83 of 360 algorithms. A trace of sixteen algorithms quantifies the risk: configuring 100 realises 91 individuals for nine of thirteen affected algorithms, as few as 31 for four that shrink, and 100 for three unaffected ones, so a table reporting 100 risks comparing populations up to 3.2-fold apart. On fifteen problems, restoring the requested size leaves the raw-IGD median no worse on thirteen (twelve better) and still better on ten after a size-matched control, yet significant on only one after correction; three repair cases demonstrated and a fourth diagnosed, so no single one-line repair is universal. The audit needs only the constructor interface, so fidelity is checkable in any such library.

Keywords population-size fidelity, reference-vector algorithm, benchmarking reproducibility, evolutionary multi-objective optimization, PlatEMO, fair comparison, large-scale optimization

Citation A population-size fidelity framework for reference-vector MOEAs. Sci China Inf Sci, for review

1 Introduction

The credibility of a comparison between multi-objective evolutionary algorithms rests on the experimental conditions it claims to hold, and one of those conditions is quietly load-bearing: the population size. Decomposition-based and reference-vector multi-objective evolutionary algorithms (MOEAs) dominate current practice, and their population is organised around a set of reference vectors whose cardinality is taken to be the population size N [1–3]. The field is moving quickly on two fronts — large language models woven into the evolutionary loop [4–6] and large-scale and many-objective problems [7, 8] — and across both one quantity is quietly load-bearing: the population size. It fixes the number of reference vectors and therefore the resolution of the search, it fixes the evaluation budget consumed per generation and therefore how many generations fit into a fixed budget, and it is reported in every experimental section as a controlled quantity. Whether a comparison is fair hinges on it, a point made explicitly in recent community guidance on fair benchmarking of evolutionary multi-objective algorithms, which names the population size among the few experimental choices that most distort a comparison [9, 10].

We report that in a widely used platform this controlled quantity is not, in general, what it appears to be, and we argue that the underlying failure mode is general enough to deserve a framework. In PlatEMO [11], a reference-vector set is obtained by calling a constructor that returns both the vectors and their number. A large number of algorithm files pass the user's population size to that constructor and then assign the second output back to the population size, so that the requested size is silently replaced by the size of the reference lattice. The constructor does not return the requested size: over the range $2 \leqslant M \leqslant 15$, $M \leqslant N \leqslant 1000$, it never returns more points than requested and returns exactly the requested number in only 8.7% of cases over that grid (about 1.6% at $M \geqslant 3$, the regime in which these methods run). At $N = 100$ the platform therefore runs with 91 individuals at three objectives and 65 at ten objectives, a reduction of 9% and 35% with no message to the user. The same mechanism can be present in any library that returns a reference set together with a size that is then written into a shared problem object.

Three quantities have to be kept apart, because the whole failure lives in the gap between them: the size a study requests and declares, N_{req}; the size the method actually evaluates, N_{real}; and the size the run records, N_{rep}. We

* Corresponding author (email: Zhangyuxuan@aeu.edu.cn)

traced all three in the platform studied here (Table 2), and the outcome is unambiguous: the constructor returns 91 for a request of 100, and nine of the thirteen affected algorithms then evaluate exactly those 91 individuals, so for those nine $N_{\mathrm{real}} = N_{\mathrm{rep}} = 91$ while $N_{\mathrm{req}} = 100$; the other four evaluate fewer. The platform always records the constructor's cardinality — never the request, which the assignment has overwritten — so for the four algorithms that shrink their own reference set during the search it does not record the size actually used either, and the recorded figure overstates the population. What is missing throughout is any signal that the request was not honoured, and the consequence is a confounded comparison rather than a corrupted result. In the same release, NSGA-II, SPEA2 and IBEA contain no such assignment and run at exactly 100, so a table that reports $N = 100$ while mixing NSGA-III with NSGA-II, or MOEA/D with SPEA2, or RVEA with IBEA, would be comparing 91, 83, 80, 66 or 31 with 100. The nominal size in the table is the same; the runs are not.

That is the failure the framework addresses, and it is a reporting failure rather than a coding mistake. The rounding itself is unavoidable: a symmetric lattice in M objectives admits only certain cardinalities, so a request that falls between two admissible values has to be rounded. What is avoidable is that the rounded value replaces the requested one without the study being told, so that the number printed in the experimental section is no longer the number the run used. It follows that the remedy is not to forbid the write-back, which many algorithms use deliberately to keep the population and the reference set consistent, but to make the divergence visible and to report the size that was realised.

The size actually realised leaves traces that a reader can check without instrumenting the platform, which is what makes the problem detectable rather than merely suspected. The per-generation cost of a reference-vector method is proportional to N_{real}, so under a fixed evaluation budget the number of completed generations is inversely proportional to it; in our controlled runs the declared size of 100 corresponds to 499 generations and the realised size of 91 to 552, a ratio of 1.106 against the predicted 1.099. A study that reports its generation count, or publishes convergence curves against generations, therefore exposes the discrepancy, and a study that reports only the final indicator hides it.

The mechanism is easy to miss, and we missed it ourselves: it surfaced only when a controlled comparison of an adaptive scheduling layer refused to reproduce. An earlier version of that layer appeared to improve its host algorithm by a double-digit percentage, and the improvement survived visual inspection of convergence curves. Most of it did not survive matching the population size, because the host was running with 91 individuals while the layer ran with 100.

We therefore present the episode not as a bug report but as the framework for *population-size fidelity* that it motivates: the property that the population size a method runs with equals the size it reports. The framework has four parts. First, a fidelity principle that states what must hold and why a violation is a threat to reproducible comparison, since reproducibility in evolutionary computation is itself an established concern [12]. Second, an auditing procedure that locates the offending constructor calls in a code base. Third, a catalogue of corrections, which shows that a single one-line repair is not universally valid and that the admissible repair depends on how an algorithm couples its population to its reference set. Fourth, a reporting protocol that makes violations visible in future work. We then apply the whole framework to PlatEMO 4.16 as a case study, quantify the consequences with controlled experiments on a large-scale suite, and show that the framework is what turns an isolated observation into a checkable, reusable procedure.

The contribution is therefore not the disclosure of a defect but the construction and validation of an instrument. Specifically, the paper (i) separates the requested, realised and reported sizes and formalises fidelity over them, proving that the deficit introduced by a lattice constructor is one-signed and inflates the number of generations completed under a fixed budget; (ii) reduces the search for infidelity to an auditable pattern, tested against fourteen constructions, twelve handled as intended and two dynamic constructions disclosed as false negatives; (iii) shows by construction that the repair is algorithm-dependent, with three cases demonstrated and a fourth diagnosed, and the mechanism of the hardest case measured rather than asserted; (iv) quantifies the consequences on a fifteen-problem suite with a paired design, effect sizes and a three-arm control that exposes the size effect at a fixed declaration together with the residual between two same-size arms; and (v) supplies a reporting protocol together with fully released materials.

2 Related work

Four strands of recent work frame this study. The first is the reference-vector family itself. Reference-vector guided selection remains the workhorse for many-objective problems, and recent methods continue to build on it, for example by adapting the reference vectors to sparse or irregular fronts and by replacing the convergence

measure used inside selection [13]. The family keeps expanding within the community, most recently with a unified dominance–decomposition framework proposed for feature selection [14]. In all of these methods the population size is the hidden constant, because the reference set and the population are indexed by the same N.

The second strand is the benchmarking-methodology literature. Community guidance has long warned that an indicator value is comparable only when the experimental protocol is fixed [15, 16], and comparison results have since been shown to depend strongly on the specification of the population size, the quantity we audit [9]; wall-clock time rather than evaluations alone has been argued to be the budget that matters in practice [17]; the reproducibility of evolutionary computation has been examined in its own right [12]; the rigour of the field has been critically reviewed [18]; and the design of benchmarks with practical impact has been reconsidered [19]. The closest antecedent to our work is an analysis of single-objective algorithms implemented across thirteen frameworks, which found that even identical parameter settings can yield different results because of implementation differences; that study raises the general concern and calls for guidelines, but it does not isolate a mechanism that a reader can check, and it covers single-objective algorithms [10]. Our contribution is a mechanism of exactly that kind, located in the reference-vector constructor, together with the audit and the repairs that make it actionable.

The third strand is the rapid arrival of methods driven by large language models, which generate offspring, design operators and steer multi-objective search end to end [4–6, 20], and are almost always benchmarked against reference-vector baselines drawn from the same platforms that we audit. A systematic population-size mismatch in those baselines would therefore propagate into the comparisons that this fast-moving literature uses to establish its gains.

The fourth strand is the widening of the problem classes to which reference-vector methods are applied: surrogate-assisted methods for expensive and high-dimensional problems, and constrained variants, are surveyed at increasing frequency [21–23], and applications built on them range from expensive optimization to real-world deployment, including work published in this journal [24, 25]. Every one of these families inherits the reference-vector selection machinery and, with it, the population-size convention under audit. The fidelity problem is therefore not tied to one benchmark but to the shared substrate of a whole class of methods.

Two positions in this literature come closest to ours and deserve an explicit contrast, because the difference is the difference between a premise and an instrument. Ishibuchi *et al.* [9] show that the population size must be specified and held fixed for a fair comparison; we show *how* a specified size can nevertheless stop being the size that is used, formalise the failure (Definition 1 and Propositions 1–2), and turn it into an audit, a repair catalogue and a reporting protocol. Ravber *et al.* [10] show that implementation differences across frameworks change single-objective results and call for guidelines; we locate one such mechanism in the reference-vector constructor of a multi-objective platform, expose its size effect at a fixed declaration together with the residual between two same-size arms through a three-arm design, and quantify the consequence rather than only raising the concern.

3 The population-size fidelity framework

We now state the framework in a form that is independent of any particular library. Throughout, a *reference-vector method* is an algorithm whose population is organised around a finite set of reference vectors, the symbol N denotes the population size that the user sets, and M denotes the number of objectives. Figure 1 summarises where the three quantities come from and where fidelity is checked.

3.1 The fidelity principle

We first make the notion precise, because the whole framework rests on distinguishing three quantities that are usually conflated.

Definition 1 (Population-size fidelity)**.** Consider a reference-vector method configured with a requested size N_{req}. At a checkpoint — the final result set, after any reference-set adaptation — the method holds a population $P = \bigcup_k P_k$ of total size $N_{\mathrm{real}} = \sum_k |P_k|$, and its output records a size N_{rep}. The method is *population-size faithful* if $N_{\mathrm{req}} = N_{\mathrm{real}} = N_{\mathrm{rep}}$ at that checkpoint; otherwise it suffers a *size infidelity*. Two drifts are worth separating. The *structural drift* $\delta_C = N_{\mathrm{req}} - L$ is the shortfall between the request and the cardinality L the reference-vector constructor returns; Proposition 1 shows it is non-negative. The *runtime drift* $\delta_R = N_{\mathrm{req}} - N_{\mathrm{real}}$ is the shortfall between the request and the population actually evaluated at the checkpoint; its sign is not fixed in general, and a method that prunes its own reference set after construction realises fewer individuals than the constructor returned (so $\delta_R > \delta_C$). Both are non-negative throughout the platform studied here.

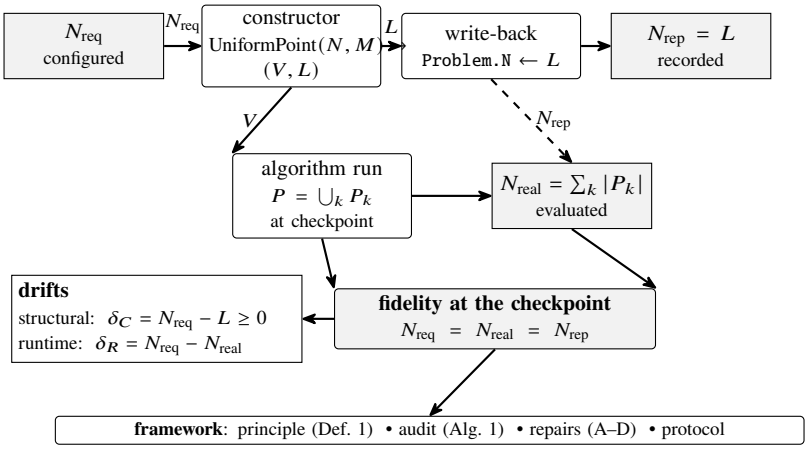


Figure 1 Where the three population-size quantities come from, and where the fidelity condition is checked. The constructor returns a reference set of cardinality L; the write-back stores L in the problem object, so the platform records $N_{\text{rep}} = L$ rather than the request; the run evaluates $N_{\text{real}} = \sum_k |P_k|$ at the checkpoint. The two drifts and the four-part framework complete the picture.

The definition isolates the three quantities deliberately, because a run can satisfy any two while violating the third. Three variants of the failure are worth separating. A run that rounds its request and then records the rounded value is honest about what it did: it is unfaithful only in the sense that the request could not be honoured, and the discrepancy is found by comparing N_{req} with N_{real} before the run rather than after it. A run that rounds, records the rounded value faithfully, and is then written up in a study whose tables state the original request is the case that threatens a comparison; there the platform is right and the paper is not, and the discrepancy is found by comparing the number in the table with the number the run realised. A run whose population changes again after the constructor has returned — because the algorithm prunes its own reference set — leaves a record that matches neither the request nor the population, and then the platform itself overstates the run. Section 4 shows that PlatEMO produces the second variant for all affected algorithms and the third variant as well for four of them. All three quantities are observable without modifying the method or the library: N_{req} is what the study configures, N_{real} follows from the cardinality of the reference set and the evaluation cost of a generation, and N_{rep} is what the run stores. This is why an audit can rest on the definition rather than on the intent of the code.

Fidelity matters for three reasons. First, the realised population fixes the cardinality of the reference set and hence the resolution of the search. Second, it fixes the evaluation budget consumed per generation and therefore the number of generations that fit into a fixed budget. Third, the reported size is the quantity a reader uses to judge whether two methods were compared under the same conditions, and this third reason is the most insidious, because an unfaithful run still produces a valid Pareto approximation and raises no error.

It is useful to separate fidelity from quality: an unfaithful run is not necessarily a poor run, and a faithful run is not necessarily a good one. Fidelity is the weaker property that the experiment is what it claims to be — exactly the property that a comparison across methods presupposes. It follows that restoring fidelity does not in itself improve an algorithm: it changes what the reported number means. The empirical sections below are written with this distinction in view, and we therefore report the direction and the size of every difference rather than treating fidelity as a performance claim.

The definition's practical interest comes from two facts that convert an anecdote into a checkable statement.

Proposition 1 (One-sided structural drift)**.** The constructor used in this study (Section 4) is the two-layer simplex lattice: a first layer of $\binom{H_1+M-1}{M-1}$ points, with H_1 the smallest integer satisfying $\binom{H_1+M}{M-1} > N$, and, when $H_1 < M$, a second layer of $\binom{H_2+M-1}{M-1}$ points with H_2 the smallest *non-negative* integer satisfying $\binom{H_1+M-1}{M-1} + \binom{H_2+M}{M-1} > N$, added only when $H_2 \geqslant 1$; the second layer is the H_2-division simplex lattice scaled by $1/2$ and shifted by $1/(2M)$ in every coordinate ("$W_2/2 + 1/(2M)$"), which is what prevents its points from duplicating the boundary directions. Then $L(N, M) = \binom{H_1+M-1}{M-1} + \binom{H_2+M-1}{M-1} \leqslant N$ for all $M \geqslant 2$, $N \geqslant M$; the structural drift $\delta_C = N - L(N, M)$ is therefore non-negative and can only remove individuals, never add them.

Proof. If $H_2 = 0$ the second layer is absent and $L = \binom{H_1+M-1}{M-1}$. By the choice of H_1, $H_1 - 1$ satisfies $\binom{(H_1-1)+M}{M-1} \leqslant N$, and $\binom{(H_1-1)+M}{M-1} = \binom{H_1+M-1}{M-1}$, so $L \leqslant N$; the boundary $M = N$ falls here with $H_1 = 1$, where the layer contributes $\binom{M}{M-1} = M = N$ rows. If $H_2 \geqslant 1$, the choice of H_2 gives that $H_2 - 1$ satisfies $\binom{H_1+M-1}{M-1} + \binom{(H_2-1)+M}{M-1} \leqslant N$, and $\binom{(H_2-1)+M}{M-1} = \binom{H_2+M-1}{M-1}$ is exactly the number of rows the second layer contributes, so $L = \binom{H_1+M-1}{M-1} +$

$\binom{H_2+M-1}{M-1} \leqslant N$. A duplicated direction between the two layers would not change the row count that $L(N, M)$ measures, so it cannot affect the inequality. The claim that the two layers never yield a duplicated direction, stated over the scanned range, is a separate empirical check, not a premise of this proof.

Equality $L(N, M) = N$ holds exactly when the request coincides with a cardinality the constructor can produce — $\binom{H_1+M-1}{M-1}$ with $H_2 = 0$, or $\binom{H_1+M-1}{M-1} + \binom{H_2+M-1}{M-1}$ with $H_2 \geqslant 1$ — which we call an *admissible* size for this constructor.

We verified the proposition exhaustively rather than trusting the argument alone. Over $2 \leqslant M \leqslant 15$ and $M \leqslant N \leqslant 1000$, that is 13895 distinct combinations, the realised count exceeds the request in 0 cases, and the two layers never yield a duplicated direction. The counts were also checked against the platform's own constructor at 39 points spanning the range and at all 81 cells of the grid reported in Table 3, with no disagreement; as regression anchors, $L(4, 3) = 3$ (no second layer is added) and $L(500, 9) = 495$ (the first layer alone exhausts the request), which pin down the $H_2 = 0$ branch that the proof handles separately.

The range is the range we scanned, not a documented limit: the platform's problems take the number of objectives as an argument, and its sources carry defaults up to ten (for example, the MaF8 problem declares `obj.M = 10` as its default). The scan extends to $M = 15$, half as much again beyond that default ceiling, which comfortably brackets the objective counts at which reference-vector methods are actually run and is cheap enough to enumerate completely; beyond it the closed form still applies at negligible cost. The complete table $L(N, M)$ over that range is released as a machine-readable file, and points outside it follow from the closed form of the constructor rather than from a measurement.

Proposition 2 (Generation inflation)**.** Consider a method that states a size $N_{\text{req}} = N$ but evaluates a population of size $N_{\text{real}} = L < N$ and consumes cL evaluations per generation, stopping when the budget $\text{FE}_{\max}$ is exhausted. If $cN \leqslant \text{FE}_{\max}$, the ratio of the generations the actual run completes to those the stated size implies is $\lfloor \text{FE}_{\max}/(cL) \rfloor / \lfloor \text{FE}_{\max}/(cN) \rfloor$, which converges to N/L as $\text{FE}_{\max} \to \infty$; the relative excess therefore approaches $(N - L)/L$, with a floor-induced error of order $O(cN/\text{FE}_{\max})$.

Proof. The number of generations is the budget divided by the per-generation cost cN_{real}. With $N_{\text{real}} = L$ the run completes $\lfloor \text{FE}_{\max}/(cL) \rfloor$ generations, whereas the stated size implies $\lfloor \text{FE}_{\max}/(cN) \rfloor$. Writing $x = \text{FE}_{\max}/c$, the ratio is $\lfloor x/L \rfloor / \lfloor x/N \rfloor$, which tends to $(x/L)/(x/N) = N/L$ as $x \to \infty$; replacing each floor by its argument incurs an error of at most $O(cN/\text{FE}_{\max})$.

The premise of Proposition 2 is the reporting condition and not the internal mechanics: the excess appears whenever the quantity a study states is N_{req} while the quantity the run realises is L. A run that both realises L and reports L is described by the same formula with $N = L$ and shows no excess, which is the consistency condition of the framework. Two consequences follow. When the constructor is the one of Section 4, Proposition 1 makes the drift one-signed, so an unfaithful method is biased in a fixed direction rather than randomly perturbed. When the comparison is made under a fixed evaluation budget, Proposition 2 makes the drift translate directly into a difference in the number of generations, which is exactly the quantity that a convergence curve displays. Neither fact is visible from a single reported population size, which is why the framework is needed.

3.2 Auditing a library for size infidelity

The audit rests on a single observation: size infidelity arises when the output of a reference-vector constructor is written into a shared object that stores the population size. Algorithm 1 formalises the procedure.

Algorithm 1 Auditing a library for population-size infidelity

Require: source tree $\mathcal{T}$; constructor set $\mathcal{C}$ with documented ranges R_ϕ of (N, M)
Ensure: census of affected call sites, files and algorithms; the deficit function $\delta_C(N, M)$
$S \leftarrow$ calls to any $\phi \in \mathcal{C}$ whose return list contains the population-size field
for each call $s \in S$ **do**
  if the pair (N, M) at s is not statically bound **then**
    mark s *unresolved*; continue
  end if
  realised deficit $\delta_s \leftarrow N - L_\phi(N, M_s)$ at the bound value (N, M_s)
  range deficit $\Delta_s \leftarrow \max_{(N,M) \in R_\phi} \left(N - L_\phi(N, M)\right)$
  if $\delta_s = 0$ **then**
    classify s as faithful; continue
  end if
  record s, its file and its algorithm directory, with δ_s and Δ_s
end for
return the census grouped by algorithm family, together with the deficit function δ_C
(run-level check) for each algorithm, compare the final $|P|$ with N_{rep}, and report any post-construction shrink

The only judgement the audit requires is the static-binding test, and the deficit evaluation uses only the documented behaviour of the constructor rather than the code that calls it, so the census is reproducible by a reader who has the same release. In the case study of Section 4 the constructor is a lattice whose cardinality is the largest admissible value not exceeding the request, which makes δ_C a deterministic function that can be tabulated once and reused; by Proposition 1 the sign of δ_C is non-negative throughout.

Two implementation choices decide whether the census is trustworthy, and we make both explicit because a naive search over source text is not. First, the match must not be satisfied by a comment or by a string literal, since algorithm files document the pattern in comments and build expressions with `eval` and `sprintf`. We therefore run the pattern over a lexically reduced form of each file in which line comments, block comments and single- and double-quoted literals have been replaced by blanks, with line and column positions preserved, and we additionally require the hit to sit on a live statement. This is a lexical and not a syntactic analysis, so it can in principle miss a call assembled dynamically. We therefore wrote a suite of fourteen positive and negative constructions, covering the constructions a MATLAB source file can use to hide such a call: comments, block comments, both quote styles, escaped quotes, a percent sign inside a string, a transpose before the match, a line continuation, an assignment to another field, and calls assembled by `eval` and by `sprintf`. Twelve behave as intended and the two dynamic constructions cannot be seen lexically at all; they are disclosed as limitations. The suite earned its keep: it showed that the parser initially missed a call split across a line continuation, which we fixed, and the census is unchanged by that fix, so the gap was a false-negative path rather than a miscount. Because a miss under-counts, we treat the census as a lower bound, and we inspected the 84 surviving hits by hand. The parser and the suite ship with the release.

Second, the audit records three quantities and not one: the requested size, the cardinality the constructor returns, and the population the run actually evaluates. Recording only the first would miss the drift, and recording only the first two would miss the four algorithms of Section 4 whose population changes again after the constructor has returned.

3.3 A catalogue of corrections

Restoring fidelity is not a single edit, because algorithms couple their population to their reference set in different ways. We distinguish four cases, and for each we state the edit, the constructor requirement and the test that fails before the edit and passes after it.

Case A: directions only. The reference vectors are used only to define directions, and the number of individuals need not equal the number of vectors. The edit discards the second return value of the constructor, `[Z,~] = UniformPoint(N,M);`, so that the reference set is left untouched while the population keeps the requested size. The applicable test is that `size(Population,1)` equals the requested N at the end of the first generation while the reference set has $L < N$ rows.

Case B: explicit indexing. The algorithm indexes its individuals by reference vector, so the population must equal the reference set. Discarding the size output breaks the indexing, and the run fails with an index-out-of-bounds error, which is the test for the case; the constructor must instead be replaced by one that returns exactly N points. A minimal exactly-N constructor is a lattice of the largest admissible size below N, padded with the $N - L$ points that maximise the minimum distance to the points already chosen, with a de-duplication step and an assertion that the number of rows equals N; because the padding is added after the lattice, the reference set changes as well as the population, which is why this repair is not interchangeable with case A.

Case C: normalisation by inter-vector geometry. The algorithm penalises a candidate by the smallest angle between neighbouring reference vectors. Here neither previous repair is admissible without additional care, and the reason can be measured. Table 1 reports the smallest pairwise angle of the two constructors. The exactly-N constructor does not collapse the angle to zero, but it reduces it, by a factor of 1.45 at $N = 100$, $M = 3$ (5.19° to 3.57°) and by a factor of 2.31 at $M = 5$ (18.19° to 7.89°). In RVEA the selection metric is the angle-penalised distance $\mathrm{APD}(F_i, V_j) = \|F_i\|\big(1 + M(\theta_{i,j}/\gamma_j)^{\alpha}\big)$ [3], in which the inter-vector angle γ_j enters the denominator of the *angular penalty*, not of the convergence term $\|F_i\|$: a smaller γ_j therefore amplifies the angular penalty and suppresses the convergence pressure along that reference vector. This is a measured candidate mechanism rather than a claim of sole cause, since the added points also change the search dynamics. A repair for this case nonetheless has to bound the angle explicitly rather than merely return N points. A minimal admissible construction is therefore: take the largest admissible lattice $L \leqslant N$, then add the $N - L$ remaining directions one at a time, each chosen to maximise the smallest angle to the directions already present, and reject a candidate whose inclusion would drop that angle below a floor $\theta_{\min} = 1°$; if the floor cannot be met, fall back to $N = L$ and report the fallback rather than silently shrinking the population.

Table 1 Smallest pairwise angle between the directions of the reference set at a requested size N. The default constructor returns $L < N$ directions; the exactly-N constructor returns N but reduces the angular resolution

N	M	default $\gamma_{\min}$	exactly-N $\gamma_{\min}$	ratio
100	3	5.19°	3.57°	1.45
100	5	18.19°	7.89°	2.31
100	8	16.38°	13.11°	1.25

We implemented this construction and measured it. At $N = 100$ with three, five and eight objectives it returns exactly 100 directions while retaining the smallest angle of the underlying lattice (5.19°, 18.19°, 16.38°), where the plain exactly-N constructor of Table 1 degrades them to 3.57°, 7.89° and 13.11°; the floor was never binding on the settings studied, so the fallback never fired (Table S4 records the three settings tested); the floor is a safety check set far below the angles that occur in practice, so its exact value does not drive the results. The applicable test is therefore two-sided and both halves are met: the minimum pairwise angle of the repaired set is at least $\theta_{\min}$, and the cardinality equals the requested N.

Case D: post-construction shrink. When an algorithm prunes or re-adapts its reference set after construction (RVEA, DGEA, LMOCSO, RVEAa in Section 4), no constructor-level repair can restore fidelity; the repair is to report the final reference-set cardinality and re-index the population to it after the last adaptation, and the test is that the recorded size equals the final population size at the checkpoint of Definition 1.

The catalogue is the reason a framework is needed at all: a reader who adopts "the one-line repair" as a universal rule will silently break the algorithms in cases B and C, and one who adopts the exactly-N constructor as a universal rule will reduce the angular resolution of those in case C by up to a factor of 2.31. Section 5.6 exhibits three of the four cases (A–C) on real algorithms; case D is diagnosed from the trace of Table 2.

3.4 A reporting protocol

Finally, the framework prescribes what a study should report — a schema rather than advice, because advice is not adoptable. A reference-vector study should print, for every configuration it compares, the seven columns

`N_req` | `N_real` | `N_rep` | `constructor` | `repair` | `generations` | `FE_used` ,

with three decision rules attached, and with the check expressed as code so that it can be run rather than remembered. Rule 1: if $N_{\text{req}} \neq N_{\text{real}}$, the row must say so, and any claim about the method must use N_{real}. Rule 2: if $N_{\text{rep}} \neq N_{\text{real}}$, the platform's record is not the population that was used, so the row must report all three quantities and no downstream claim may rely on the record; Table 2 shows that four of the sixteen traced algorithms are in this state. Rule 3: a comparison table must not mix rows whose `N_real` differ unless the difference is the object of study, which Table 2 shows is violated by default. The protocol costs one line in the experimental section and one assertion at the end of a run, `assert(numel(Population) == N_real && Problem.N == N_real)`. The protocol deliberately does not require suppressing the write-back of the size into the shared problem object, which many algorithms use to keep the population and the reference set consistent. What it requires is that the number in the table be the number the run used, and that an unrealisable request be declared as such before the runs rather than discovered afterwards. A study that cannot realise a request should either move the request to the nearest admissible value, as we do for the declared arm of Section 5, or state the realised value and treat the request as nominal.

4 Case study: auditing PlatEMO

4.1 The override

We audit PlatEMO 4.16, released in July 2026 with 360 algorithms and 630 problems — the version that added the host of Section 5, and the reference point for the census below. The affected line is

```
[Z,Problem.N] = UniformPoint(Problem.N,Problem.M);
```

in which the second output of the constructor [26] is written into the population size. The constructor is the simplex lattice of Das and Dennis [27] extended to a second, shifted inner layer, as introduced by the reference-point generation of NSGA-III [1]; it caps the total cardinality at the request, which is an implementation choice rather than a mathematical necessity, and the second layer is itself a fidelity effort that shrinks the gap between request and realisation. What we examine is the write-back: the capped value is assigned to the field that stores the

Table 2 Trace of the three quantities for 16 reference-vector algorithms of PlatEMO 4.16, all configured with $N_{\mathrm{req}} = 100$ at $M = 3$. L is the cardinality returned by the constructor, N_{real} the population present in the run's own result, and N_{rep} the value held in the platform's population-size field after the run. N_{rep} equals L for each of the 13 affected algorithms, so the recorded value is the constructor's cardinality rather than the realised population wherever the two differ. The last three algorithms carry no such assignment and serve as controls

algorithm	N_{req}	L	N_{real}	N_{rep}	status
NSGA-III	100	91	91	91	diverges
MOEA/D	100	91	91	91	diverges
RVEA	100	91	83	91	diverges
LMOCSO	100	91	66	91	diverges
FDSEA	100	91	91	91	diverges
HEA	100	91	91	91	diverges
C-TAEA	100	91	91	91	diverges
MOEA/D-DRA	100	91	91	91	diverges
MOEA/D-DE	100	91	91	91	diverges
RVEAa	100	91	31	91	diverges
DGEA	100	91	80	91	diverges
PPS	100	91	91	91	diverges
MOEA/D-STM	100	91	91	91	diverges
NSGA-II	100	–	100	100	faithful
SPEA2	100	–	100	100	faithful
IBEA	100	–	100	100	faithful

Table 3 Realised population size returned by the default reference-vector constructor for requested sizes N and objective counts M. Entries equal to the request are set in bold. The constructor never exceeds the request and equals it only at admissible lattice sizes

$M \setminus N$	91	100	105	120	150	200	275	300	500
2	**91**	**100**	**105**	**120**	**150**	**200**	**275**	**300**	**500**
3	**91**	91	**105**	**120**	136	190	253	**300**	496
4	84	84	84	**120**	120	165	220	286	455
5	85	85	**105**	105	126	126	210	210	495
6	77	77	77	112	147	182	273	273	462
7	**91**	91	91	112	112	168	238	294	490
8	72	72	72	**120**	128	156	240	240	450
9	90	90	90	90	90	174	210	210	495
10	65	65	65	110	110	110	**275**	275	440

user's request. Table 2 traces the three quantities for 16 reference-vector algorithms at a requested size of 100 and three objectives. Two facts follow. First, because `PROBLEM` is a handle class, the assignment mutates the caller's object, so the platform records the constructor's cardinality (91) and never the request; it records L exactly — faithful to the constructor, not to the request. Second, that recorded value is not always the population used: nine of the thirteen affected algorithms coincide at 91, but RVEA finishes at 83 individuals, DGEA at 80, LMOCSO at 66 and RVEAa at 31 (factors 1.10, 1.14, 1.38 and 2.94 over the recorded value), while NSGA-II, SPEA2 and IBEA are unaffected and run at the requested 100, so a study comparing any two at a nominal $N = 100$ compares populations differing by up to a factor of 3.2.

4.2 How the affected files use the write-back

The lexical audit of Section 3.2 also records what happens to the two outputs in each affected file. In 42 of the 84 affected files the returned set is used again after the assignment, most often as the reference set of environmental selection, so population, reference set and per-generation cost change together; in the rest only the size field is overwritten. The assignment precedes population initialisation in 62 of the files (74%) and follows it in 6 (FDV, GLMO, WASF-GA, GWASF-GA, LCSA and POCEA); in the remaining 16 the initialisation sits in a helper the audit does not resolve. The released per-file table gives the line numbers, so a reader can check each classification.

4.3 Realised count versus requested size

Table 3 and Figure 2 give the realised count for requested sizes up to 500 and objective counts from 2 to 10. The deficit is zero only when the request coincides with an admissible lattice size. At $N = 100$ and $M = 3$, the setting used in the great majority of three-objective studies, the algorithm runs with 91 individuals; at $M = 5$ with 85; at $M = 8$ with 72; at $M = 10$ with 65. The worst deficit in the table is 45%, and over the whole scanned range the worst case is larger still, reaching a deficit of 75% of the request — that is, realising as little as a quarter of what was asked — at high objective counts; the released table covers that range.

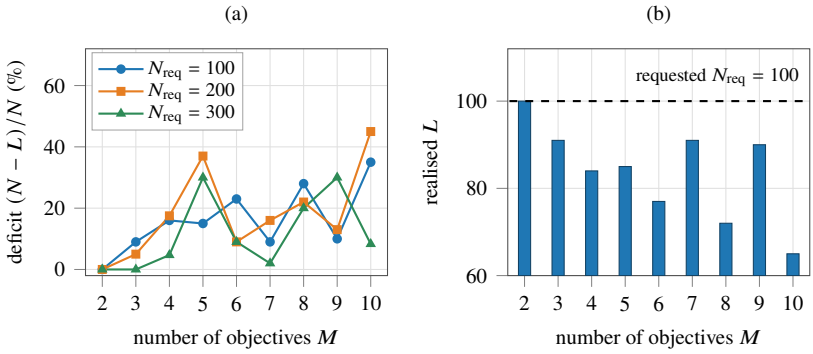


Figure 2 The default reference-vector constructor of PlatEMO 4.16. (a) Deficit $(N - L)/N$ as a percentage against the number of objectives for three requested sizes; the deficit is non-monotone in M and is zero exactly at admissible sizes. (b) Realised count L for a request of $N = 100$ across objective counts, against the requested 100 (dashed); the y-axis starts at 60.

Table 4 Census of the PlatEMO 4.16 algorithm tree, at both the algorithm and the file level, for the assignment `[Z,Problem.N] = UniformPoint(Problem.N,Problem.M)`. The algorithm count is the number of directories containing at least one affected file

family	algorithms	files	examples
MOEA/D family	26	26	MOEA/D, MOEA/D-DRA, MOEA/D-DE, ...
NSGA-III family	4	4	NSGA-III, A-NSGA-III, DCNSGA-III, ...
RVEA family	4	4	RVEA, RVEAa, RVEA-iGNG, ...
Large-scale	14	15	LMOCSO, HEA, C-TAEA, FDSEA
Other	35	35	AdaW, ParEGO, SPEA-R, ...
total	83	84	of 360 algorithms, 1887 files

4.4 Census of the distribution

We searched the released tree. The platform ships 360 algorithms and 630 problems, organised as 360 algorithm directories, one per algorithm, holding 1887 `.m` source files; a single algorithm is split across a main file and its helper files, so the census reports both levels. Of the 1887 files, 84 (4.5%) contain the assignment, and they belong to 83 of the 360 algorithm directories, that is 23.1% of the algorithms in the release; because the lexical audit of Section 3.2 under-counts by construction, both figures are lower bounds. 82 of the 83 contribute one file each and one algorithm contributes two, so $82 \times 1 + 1 \times 2 = 84$ files across 83 algorithms. The affected algorithms include the canonical NSGA-III and MOEA/D implementations and their descendants, RVEA and its variants, LMOCSO, HEA and C-TAEA, and the host itself, while 3 of the sixteen algorithms we traced (NSGA-II, SPEA2, IBEA) are unaffected and run at the requested size, which is what makes the comparison in Table 2 a confounded one. Table 4 groups the affected algorithms by family. The assignment is not a rare idiom confined to one author, which is why the framework rather than the incident is the contribution.

4.5 The same request in another library

The framework claims to be portable, so we ran the audit on a second library rather than asserting that it would apply. pymoo is a widely used Python framework with a reference-direction module of its own, and it exposes the same two constructions: a Das–Dennis lattice and a factory that returns an exact number of directions. The audit's trigger, a routine that returns a reference set together with a size that is then written into a shared population-size field, is *absent* there: the Das–Dennis entry point takes the partition count and derives the cardinality, and the exact-count factory is a separate call, so no user-set population size is overwritten. The audit therefore returns a negative result on pymoo [28], which is what makes it a test rather than a tautology.

The comparison in Table 5 is therefore not a second defect report. What is *not* absent is the underlying arithmetic, and the difference between the two libraries is instructive: the table compares the realised cardinality for the same nominal request. At three objectives both libraries return 91, because the lattice is the same; at five, eight and ten objectives they diverge, by factors of 1.21, 2.00 and 1.18. At eight objectives a study that declares $N = 100$ realises 72 directions in PlatEMO and 36 in pymoo, a factor of 2.00. The divergence is not a defect of either library: PlatEMO returns a two-layer lattice that caps the total at the request, while pymoo selects the largest

Table 5 Realised cardinality of the reference set at a nominal target of 100 directions (and 200 in the last column) in two libraries. The first row gives the nominal target, the second the number of objectives; the last row is the PlatEMO value divided by the pymoo value. PlatEMO returns a two-layer lattice capped at the target, pymoo the largest single-layer lattice not exceeding it, so the ratio describes a difference in construction, not a defect of either

library	100	100	100	100	200
(N_{req})	3	5	8	10	3
PlatEMO 4.16	91	85	72	65	190
pymoo 0.6.2	91	70	36	55	190
ratio	1.00	1.21	2.00	1.18	1.00

single-layer lattice whose cardinality does not exceed the request and stops there; the two constructions differ, so the ratio reflects that difference, not a shortcoming of either. The negative result on pymoo concerns the *idiom* — the write-back of a size alongside a reference set — being absent, not the absence of infidelity of any kind. The point for the framework is that the nominal size does not determine the realised size even across two libraries, let alone across algorithms within one library, and that a comparison table reporting only the nominal size is therefore unverifiable by construction.

5 Empirical consequences of the override

5.1 Experimental setup

The host is FDSEA [29], a recent large-scale reference-vector method that contains the assignment, and it couples its population to the reference set only through the directions, so it is case A of Section 3.3. Its published generation loop is governed by three constants: the number of individuals exchanged between its two subpopulations (n_{ex}, published default 10), the hybridisation weight that mixes a genetic and a differential operator (γ, published default 0.5 under an online update rule), and the neighbourhood size of the frequency-domain subpopulation (K, published default 5). We hold K at 5 and set the other two statically, so that the effect of the population size is separated from the effect of the constants; the arm labelled "published" below therefore means the published constants held static, not a literal re-run of the online update rule, which we freeze for that isolation. We compare nine configurations of the same host, all single-threaded, all with an evaluation budget of 10^5 and seeds $1, \ldots, 30$ for the $D = 300$ problems and 20 for the $M = 5$ and $D = 1000$ problems: the host as published; the host with the size output discarded (repair 1); the host with the exactly-N constructor (repair 2); an adaptive scheduling layer that updates n_{ex}, γ and K online from a convergence and a diversity signal, used later only as a case study; and five static settings of n_{ex} and γ. They appear, respectively, in Table 6 (published and repair 1 — there named "repaired" — alongside the declared control) and, in the supporting information, Tables S3 (repair 2), S5 (layer), S1 (static settings) and S2 (the separate K sweep). The suite has 15 problems: DTLZ1, DTLZ2 and DTLZ7 [30] and the nine instances LSMOP1–LSMOP9 [31] at $M = 3$, $D = 300$; LSMOP1 and LSMOP6 at $M = 5$, $D = 500$; and LSMOP1 at $M = 3$, $D = 1000$. The indicator is the inverted generational distance [32] to a 400-point discretisation of the true front; for every front used here the front spans the unit interval in each objective, so the usual normalisation is the identity.

Two points about the design determine how the results are to be read. First, the comparison isolates something narrower than the framework as a whole. All nine configurations are launched with the same requested size $N = 100$, and Section 5.2 adds a further declared arm that exposes the size effect at a fixed declaration together with the residual between two same-size arms; "as published" realises $N_{\mathrm{real}} = 91$ because the override takes effect, and "repair 1" realises $N_{\mathrm{real}} = 100$. The measured difference is therefore the difference between the declared condition and the realised one, and by construction it coincides with the difference between a 91-individual and a 100-individual run of the host. We do not claim that fidelity has an effect separable from the population size; we claim, and measure, that declaring one size while running another changes the conclusion a study would draw. Second, the termination test is evaluated at generation boundaries, so a run can overshoot the budget by less than one generation: the published arm consumes 100090 evaluations (0.09% above nominal) and the corrected arms 100000 exactly.

Algorithms share seeds and the runs are treated as paired, so we use the two-sided Wilcoxon signed-rank test, reporting also the Holm–Bonferroni correction over the 15 comparisons and the sign test. We do not verify stream alignment, and the residual of Section 5.2 shows that same-seed runs are not bit-identical, so the paired p-values should be read with that caveat (an unpaired permutation check is left to the released records). Throughout, the central tendency is reported as the *median* over the runs, the interquartile range of the per-run record is used as spread, and every percentage change quoted in the text and in the tables is computed from medians as a ratio of medians, so that the two agree; optimisation runs occasionally produce a few dominating outliers, and the median

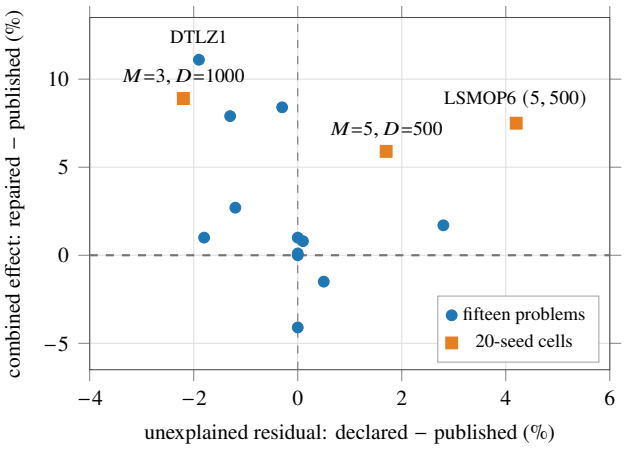


Figure 3 The two contrasts of the three-arm design, one point per problem. The horizontal axis is the unexplained residual between the two same-size arms (declared minus published); the vertical axis is the combined effect (repaired minus published); the dashed lines mark zero. The three 20-seed cells (M=5 and D=1000) are drawn as squares

is the robust summary. The raw per-run records are released, so the mean can be recovered exactly.

5.2 Three arms: published, declared and repaired

Because the published configuration differs from the repaired one in the population actually evaluated, a comparison between those two alone cannot say whether the problem is the declaration or the size. We therefore ran a third arm in which the host is configured at the size it will realise — 91 at three objectives and 85 at five, which are the two values the constructor returns for a request of 100 at those objective counts (Table 3) — so that the declaration and the realisation agree. The three arms are then *published* (declared 100, realised 91), *declared* (declared 91, realised 91) and *repaired* (declared 100, realised 100). Table 6 gives all three pairwise comparisons, and each isolates a different thing: declared against published holds the realised size fixed at 91 and varies only the declaration, so it measures the residual between two same-size arms (the write-back is deterministic, so there is no declaration effect to measure); repaired against declared varies the realised size from 91 to 100 at a fixed declaration, so it measures the *size* effect; and repaired against published is their combination, which is the quantity a reader of the published paper actually experiences.

The declared arm requests 91 at three objectives and 85 at five, which are the values the published arm realises; the two arms therefore evaluate the same number of individuals. In the host's source the assignment `[Z,Problem.N] = UniformPoint(Problem.N,Problem.M)` precedes population initialisation, so for a fixed request the constructor returns the same reference set regardless of whether the request is 100 or its realised value, and the two arms should be near-identical. Their *signed* median change is +0.0%, the median of the absolute changes is 1.2% and the largest is 4.2%, and the paired signed-rank test separates none of the 15 problems: the smallest of the 15 p-values is 0.12 (Table S6 lists them). This arm is a manipulation check, but an inconclusive one as to the residual's source: it confirms that both arms evaluate the same 91 individuals, yet under the official source the two arms ought to be bit-identical while they differ by a median 1.2% and a maximum 4.2%. We report this as an *unexplained residual* — a seed-stream effect or a harness-level difference — and flag it for tracing rather than attribute it to a declaration effect. Against that, repaired and declared differ by a median of 2.7%, and because repair 1 leaves the reference set at its 91 directions while raising the population to 100, this measures the effect of the population count together with the resulting 100-to-91 ratio between population and reference set; we call it the size effect with that caveat. Read against the published arm instead, the repaired host is better on 12 problems, identical on 1 and worse on 2, with a median of +1.7%; that is the count the abstract reports. The three arms therefore expose the size effect at a fixed declaration together with the unexplained residual between the two same-size arms, and they locate the defect where the framework puts it: not in the algorithm, which behaves as the realised size dictates, but in the number printed beside the result. Figure 3 plots the two contrasts against each other: the residual is centred on zero while the size effect is the load-bearing signal.

Table 6 Three-arm comparison: published, declared and repaired, on the same host and seeds. Published declares 100 and realises 91; declared declares and realises 91 (the request is 91 at three objectives and 85 at five); repaired declares and realises 100. The three right-hand columns are median relative changes of the first-named arm against the second, positive meaning the first attains a smaller indicator, so that $decl - pub$ measures the residual between two arms that realise the same population, $rep - decl$ the size effect at a fixed declaration, and $rep - pub$ the combined (size plus residual) effect

problem	published	declared	repaired	decl−pub	rep−decl	rep−pub
DTLZ1	0.0657	0.0670	0.0584	−1.9	+12.8	+11.1
DTLZ2	0.0609	0.0620	0.0603	−1.8	+2.7	+1.0
DTLZ7	0.0590	0.0598	0.0543	−1.3	+9.1	+7.9
LSMOP1	0.1032	0.1035	0.0945	−0.3	+8.7	+8.4
LSMOP2	0.0812	0.0807	0.0823	+0.5	−2.0	−1.5
LSMOP3	0.8645	0.8645	0.8645	+0.0	+0.0	+0.0
LSMOP4	0.1448	0.1408	0.1424	+2.8	−1.2	+1.7
LSMOP5	0.3559	0.3554	0.3531	+0.1	+0.7	+0.8
LSMOP6	0.6429	0.6428	0.6363	+0.0	+1.0	+1.0
LSMOP7	0.8052	0.8052	0.8383	+0.0	−4.1	−4.1
LSMOP8	0.0769	0.0778	0.0748	−1.2	+3.9	+2.7
LSMOP9	0.7359	0.7358	0.7351	+0.0	+0.1	+0.1
LSMOP1 (5,500)	0.3971	0.3904	0.3736	+1.7	+4.3	+5.9
LSMOP6 (5,500)	0.9400	0.9001	0.8693	+4.2	+3.4	+7.5
LSMOP1 (3,1000)	0.0992	0.1013	0.0904	−2.2	+10.8	+8.9

5.3 Effect of restoring fidelity on the host

Table 6 reports the host on all 15 problems, Table S3 of the supporting information adds repair 2, and Figure 4 shows the per-problem median change. Restoring the requested size changes the median indicator from -4.1% to 11.1%, the median change over the suite being 1.7%. The direction of the effect is not uniform, and we state the counts exactly rather than in summary. These counts are against the published arm — the $rep - pub$ column of Table 6 — which measures the combined (size plus residual) effect; the size effect ($rep - decl$) is reported in Section 5.2. The corrected host is no worse on 13 of the 15 problems and worse on the remaining 2. We give the breakdown rather than a single count: twelve of the 13 improve in the same direction (eight of them above 1%), LSMOP3 is a tie (the medians differ by about 3×10^{-6}, so the printed value is +0.0%), and the two degradations are LSMOP2 and LSMOP7 (by −1.5% and −4.1% respectively). The size effect ($rep - decl$ in Table 6, the population count changed while the reference set stays at its 91 directions) degrades three problems: LSMOP2, LSMOP4 and LSMOP7; in the combined count LSMOP4 instead improves by +1.7%, because its positive residual (+2.8%) offsets its negative size effect (−1.2%). This is Proposition 2's fixed-budget trade-off in miniature: the extra nine individuals buy wider coverage and cost about a tenth of the generations, and on those three problems the lost generations outweigh the added coverage. Read as *no worse*, the count is thirteen; read as *improved*, it is twelve, and we state the count as no-worse because four of the twelve improvements are within 1%.

Statistical significance is weaker than the direction count. The change is significant on 4 of the 15 problems at the uncorrected 0.05 level, and on 1 problem after Holm–Bonferroni correction over the 15 comparisons; the single survivor is an $M = 5$ instance, where the realised deficit is largest and the cell has twenty seeds rather than thirty. Power over fifteen comparisons is limited, so we report the win/loss counts as a distribution-free summary (Table 7). Independently, a bootstrap confidence interval for the median difference excludes zero on 8 of the 15 problems. Because that interval is likewise uncorrected for multiplicity, its count is best read against the 4 uncorrected tests rather than the single Holm-corrected survivor; we report it as exploratory and lead with the corrected count. The honest reading: the direction of the effect is consistent across the suite, its magnitude is confidently non-zero on about half the problems, and Proposition 1 bounds the sign rather than the size.

Two points deserve emphasis. First, the measured difference is a size effect as much as a fidelity effect: the override makes the host run with fewer individuals than its reported size, so a study that prints $N = 100$ and runs 91 individuals publishes a number that belongs to a smaller configuration. Because the inverted generational distance is non-increasing in the number of returned points, part of the gain is mechanical, and we separate it by downsampling each repaired front to the published arm's realised size (91 at three objectives, 85 at five) and recomputing the indicator against the same reference front. We draw random subsets rather than a coverage-maximising one, because a spread-maximising selection optimises the distance between the selected points rather than the distance to the reference front that this indicator averages, and can select outliers that raise it. Under the median of 200 random subsets (seed 0) the median gain falls from 1.7% to 0.8%, and the number of problems on which the repaired host is better falls from twelve to ten (Table S8 gives the per-problem values). The surviving gains are concentrated on DTLZ1 (7.0%), LSMOP1 (6.2%), LSMOP6 at five objectives (7.0%) and LSMOP1 at

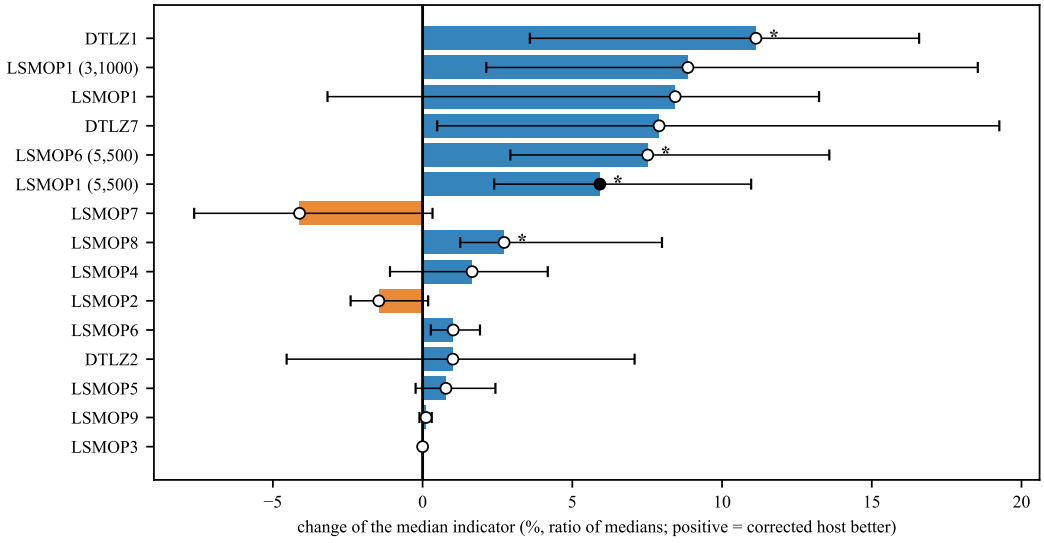


Figure 4 Change of the median indicator of the host from restoring the requested population size, one bar per problem, sorted by the magnitude of the change. The bar is the ratio of medians, the same quantity as the $rep - pub$ column of Table 6; blue marks a better (smaller-indicator) corrected host and orange a worse one, the error bar is a bootstrap 95% confidence interval for the median difference, an asterisk marks uncorrected $p < 0.05$, and the solid marker is the single Holm-corrected survivor (Table 7).

Table 7 Wilcoxon signed-rank test of repair 1 against the published host. Section 5.2 shows the declared arm differs from the published run only by a small unexplained residual (the paired signed-rank test separates none of the 15 problems; Table S6), so this table measures the combined (size plus residual) effect. "Repair 1 wins" counts the runs on which the corrected host attains a smaller indicator

problem	repair 1 wins	p (signed rank)	Holm–Bonferroni	sign test
DTLZ1	21/30	0.0155	0.1855	0.0428
DTLZ2	18/30	0.3387	1.0000	0.3616
DTLZ7	20/30	0.0879	0.7914	0.0987
LSMOP1	18/30	0.0549	0.6041	0.3616
LSMOP2	12/30	0.1840	1.0000	0.3616
LSMOP3	14/30	0.7151	1.0000	0.8555
LSMOP4	15/30	0.4522	1.0000	1.0000
LSMOP5	19/30	0.4400	1.0000	0.2005
LSMOP6	19/30	0.0699	0.6989	0.2005
LSMOP7	15/30	0.3931	1.0000	1.0000
LSMOP8	22/30	0.0099	0.1291	0.0161
LSMOP9	17/30	0.8078	1.0000	0.5847
LSMOP1 (5,500)	19/20	$< 10^{-4}$	$2.9e-04$	$< 10^{-4}$
LSMOP6 (5,500)	17/20	0.0037	0.0512	0.0026
LSMOP1 (3,1000)	14/20	0.0973	0.7914	0.1153

D=1000 (6.4%). Most of the median gain is therefore mechanical, but a small genuine effect remains on ten of the fifteen problems. Second, the direction of the mean can disagree with the median and with the majority of runs when a few outliers dominate: on DTLZ7 the corrected host is better on 20 of the 30 runs and by median, yet its mean is worse — which is why a single mean hides the effect.

5.4 Two ways of restoring fidelity

Section 5.2 used the conservative repair to expose the size effect at a fixed declaration. Table S3 of the supporting information gives repair 2 alongside repair 1; the two are not equivalent, because repair 2 changes the reference set as well as the population, and on several problems the default constructor's smaller reference set is at least as good. Where the algorithm permits the population to differ from the reference set, repair 1 is the conservative choice.

5.5 A reported gain undone by the override

To show how the override manufactures a gain we use the adaptive scheduling layer that originally motivated this work; it drives n_{ex}, γ and K online from a convergence and a diversity signal, and it is a case study, not a contribution (Table S5). Against the published host the layer is better on 10 problems and worse on 5, with a median gain of only +0.87%: the gains above 5% that motivated it are concentrated in 6 problems. Against the host at the requested population size the picture narrows to 8 better and 7 worse, median +0.04%. On the 6 problems carrying a gain

Table 8 NSGA-III with the published override and with the requested population size restored by repair 1, on six large-scale problems. NSGA-III is case A, so repair 1 leaves the reference set untouched; the realised population is 91 in the published row and 100 after the repair, as in Table 6

NSGA-III (median)	LSMOP1	LSMOP2	LSMOP4	LSMOP6	LSMOP8	LSMOP9
as published	0.6288	0.0829	0.1963	2.9299	0.3459	1.0834
repair 1 (requested N)	0.5832	0.0836	0.1999	3.5033	0.3437	1.0770
p (signed rank)	0.0841	0.0155	$6.7e-04$	0.5291	0.9354	0.0767

above 5%, matching the size removes 0.9 to 8.8 percentage points of it, so much of the apparent advantage is the population difference rather than the scheduling; against the published host, the remaining nine stay within 8.6% in either direction, with four in the layer's favour and five against. Against the best static setting the layer attains a smaller median on only 2 of the 15 problems. We therefore do not claim that the layer is worthless; we claim that the evidence which previously supported it was largely an artefact of comparing a 100-individual layer against a 91-individual host, and that at a matched size the advantage is small and inconsistent in sign.

5.6 Consequence across algorithms

The affected set is not confined to one family. A comparison table that mixes affected and unaffected methods is therefore confounded, because the affected methods run with fewer individuals than they report. Table 8 quantifies this on NSGA-III. The picture is more equivocal than for the host: the corrected NSGA-III is better on LSMOP1 but worse on LSMOP4 and LSMOP6, and the signed-rank test is significant on LSMOP2 and LSMOP4 at the uncorrected 0.05 level even where the medians barely move, because the override also changes the shape of the run-to-run distribution rather than only its centre; after Holm correction over the six comparisons of Table 8 only LSMOP4 remains significant. There is accordingly no uniform sign to the bias.

We attempted the same repair on the other affected peers, where the catalogue of Section 3.3 becomes concrete. NSGA-III is case A and accepts repair 1. MOEA/D is case B: it pairs each weight vector with a neighbourhood over the reference set, so the population must match the reference set; repair 1 fails with an index-out-of-bounds error, while repair 2 restores it. RVEA and LMOCSO [33] are case C: both compute the smallest inter-vector angle $\gamma_i = \min_j \arccos(1 - \mathrm{d}_{\cos}(v_i, v_j))$, with $\mathrm{d}_{\cos}$ the cosine distance, and both use γ_i in the denominator of the angular-penalty factor of their angle-penalised distance, $\mathrm{APD}(F_i, V_j) = \|F_i\|\big(1 + M(\theta_{i,j}/\gamma_j)^{\alpha}\big)$ [3]. The mechanism is the one measured in Table 1: the exactly-N constructor reduces the smallest angle by a factor of 1.45 at $M = 3$ and 2.31 at $M = 5$, so γ_i falls and the angular penalty is amplified, suppressing the convergence pressure along that reference vector — a quantitative degradation, and the reason the case needs its own repair; we offer this as the measured candidate mechanism rather than the only one, since the added points also alter the search dynamics. Repair 1 fails for RVEA as well, though later than MOEA/D's immediate failure: RVEA runs until the first reference-vector adaptation, which the platform schedules at $f_r = 0.1$ of the planned generations, where the assignment $V(1:N,:)$ requires the vector set to have at least N rows. Table S7 of the supporting information reports the catalogue validation on DTLZ1–7 at three objectives (PlatEMO's DTLZ defaults, requested N=100, the budget of Section 5). Repair 2 restores MOEA/D, which runs at exactly 100 individuals throughout; Table S7 reports the median IGD of each variant, and we do not tabulate p-values there, so the per-seed records from which they can be computed are released.

On the degenerate fronts DTLZ5–6 the unbounded constructor degrades the Case C pairs (RVEA's median IGD more than triples, LMOCSO's by over an order of magnitude), while the bounded-angle constructor repairs the damage — for RVEA it is better than the published arm on all seven problems (Table S7). The bounded-angle repair is validated on the DTLZ family; its behaviour on the large-scale LSMOP suite is left to the released scripts and to future work.

Fidelity can accordingly be restored everywhere but not uniformly: the admissible repair follows from the coupling between population and reference set, not from a universal rule.

5.7 Static settings and cost

Sweeping the two constants statically (Table S1 of the supporting information) suggests that the exchange count is the load-bearing one, and it provides the reference against which the layer of Section 5.5 is judged; it is context for the case study only, and therefore lives in the supporting information. The host as published completes 552 generations and the corrected host 499, a ratio of 1.106 against the ratio of realised population sizes, 1.099, as

Table 9 Population, evaluations, generations and wall-clock time of the two arms of Section 5.2. "FE / generation" is the steady-state evaluations per generation, computed as (evaluations $- I$)/generations with I the one-time evaluation of the initial population ($I = 91$ and 100), so each row closes as FE $= I +$ generations $\times$ (FE/generation). The K sweep (Table S2) is recorded at exactly 10^5 evaluations per run and is reported in the supporting information

configuration	population	evaluations	generations	FE / generation	wall (s)
as published	91	100090	552	181.16	18.9
requested N	100	100000	499	200.20	18.5

Proposition 2 predicts in its asymptotic form. The account is not exact, and we state it rather than paper over it. Table 9 gives the raw figures, which close as FE $= I + G \times g$, where I is the one-time evaluation of the initial population (91 for the published arm, 100 for the corrected arm) and g the steady-state cost per generation: for the published arm $g = (100090 - 91)/552 \approx 181.16$ and for the corrected arm $g = (100000 - 100)/499 \approx 200.20$. The cost per individual is therefore ≈ 1.99 and ≈ 2.00 rather than a constant 2, and this drift in the per-individual cost is what separates the observed ratio 1.106 from the predicted 1.099; the 90-evaluation overshoot (0.09%) is only a small part of it. Because each arm has a single run, we present this as a consistency check of Proposition 2, not a verification of it. The wall-clock differences follow from the same effect and are not evidence about the algorithm.

The same static family checks that the comparison between the layer and the static settings of Section 5.5 is made over the same search space: the layer updates K online, the static settings held it at its published value, and Table S2 of the supporting information closes that gap by sweeping $K \in \{3, 5, 10\}$ at the fixed setting n_{ex}=10, γ=0.7. The sensitivity is not negligible, though several best-value counts rest on sub-1% margins: the best value differs across problems (5 prefer K=3, 3 prefer K=5, 7 prefer K=10), and on DTLZ1 the setting K=10 fails to converge — a median indicator of 1116.0 against 0.0418 for K=3, with 29 of the 30 runs above unity, so a static choice of K cannot be treated as neutral. The detail lives in the supporting information, since this is context for the case study only.

6 Discussion and conclusion

We have argued that the population size of a reference-vector MOEA is a controlled quantity that can silently stop being controlled, and we have given a framework that makes the failure detectable, repairable and reportable. Applied to PlatEMO 4.16, the framework finds the assignment in at least 84 of 1887 source files across at least 83 of 360 algorithms, and on a large-scale suite restoring the requested size leaves the host's raw-IGD median no worse on thirteen of fifteen problems, and better on ten after a size-matched control, with limited significance (one Holm-corrected survivor); it accounts for most of the median scheduling gain that had been attributed to the scheduler, and it changes sign across peers.

Two methodological conclusions follow. The first is that the framework converts a diffuse worry about experimental fairness into a procedure with four checkable outputs: an audit yielding a census at the algorithm and file level, a deficit table, a repair catalogue with the mechanism of the hardest case measured, and a reporting protocol. The second is that the finding should be read as a reporting concern rather than a defect in the algorithms: the rounding of a simplex lattice is unavoidable and the write-back is often deliberate; what is avoidable is publishing a table that states a size the run did not use. The trigger — a constructor that returns a reference set together with a size written into a shared field — is not specific to PlatEMO, and the audit is stated language-neutrally, so any library with this pattern can be audited in the same way.

The study has clear limits. The census measures one distribution at one version and is not a survey; we demonstrate the comparison risk in a controlled setting but do not survey the published literature, so how often published studies report the requested rather than the realised size remains open. The declared-versus-realised comparison coincides by construction with a difference in population size, so it is the size-matched control of Section 5.3 that separates the mechanical from the search component, and the residual effect it leaves is small. The $M = 5$ and $D = 1000$ cells use twenty seeds, and power is limited over fifteen comparisons. The framework detects and repairs infidelity but cannot decide how large a population should be.

Data availability. The release is publicly available at https://github.com/qingdi-miruisi/population-size-fidelity, at tag v1.24, which pins one immutable commit, and contains the audit script of Algorithm 1, the per-file audit listing, the realised-count table $L(N, M)$ over 13895 combinations, the trace and cross-library tables, the corrected variants for the four repair cases, the driver scripts that regenerate every table and figure, and the raw per-run archives from which all reported statistics are recomputed. No file of the audited distribution was modified; all experiments ran in MATLAB R2026a against PlatEMO 4.16 with the recorded seeds and are single-threaded.

Conflict of interest. The author declares that he has no conflict of interest: the audited platform is an open-source

community project with which he has no financial or personal relationship.

Author contributions. The sole author, Y. X. Zhang, was responsible for all aspects of the work.

Acknowledgements This research received no external funding. The author thanks the developers of PlatEMO for making the platform and its source code openly available, which is what makes an audit of this kind possible.

Supporting information Tables S1–S8: static settings; K sensitivity; repair 2 alongside repair 1; the bounded-angle constructor; the scheduling layer against both references; p-values; the catalogue validation on DTLZ1–7; the size-matched IGD control. Also the parser test suite, the per-file audit listing, the realised-count table, the measured angles, the corrected variants and the raw per-run records. The supporting information is available online at info.scichina.com and link.springer.com. The supporting materials are published as submitted, without typesetting or editing. The responsibility for scientific accuracy and content remains entirely with the authors.

References

1 Deb K, Jain H. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. IEEE Trans Evol Comput, 2014, 18: 577–601
2 Zhang Q, Li H. MOEA/D: a multiobjective evolutionary algorithm based on decomposition. IEEE Trans Evol Comput, 2007, 11: 712–731
3 Cheng R, Jin Y, Olhofer M, et al. A reference vector guided evolutionary algorithm for many-objective optimization. IEEE Trans Evol Comput, 2016, 20: 773–791
4 Wu X, Wu S H, Wu J, et al. Evolutionary computation in the era of large language model: survey and roadmap. IEEE Trans Evol Comput, 2025, 29: 534–554
5 Liu F, Lin X, Yao S, et al. Large language model for multiobjective evolutionary optimization. In: Proceedings of the International Conference on Evolutionary Multi-Criterion Optimization, Singapore, 2025. 178–191
6 Huang Y, Wu S, Zhang W, et al. Autonomous multi-objective optimization using large language model. IEEE Trans Evol Comput, 2026, 30: 594–608
7 Liu J, Sarker R, Elsayed S, et al. Large-scale evolutionary optimization: a review and comparative study. Swarm Evol Comput, 2024, 85: 101466
8 Wang P, Wu X, Deng H. Large-scale multi-objective optimization algorithms: a decade survey. Expert Syst, 2025, 42: e70157
9 Ishibuchi H, Pang L M, Shang K. Population size specification for fair comparison of multi-objective evolutionary algorithms. In: Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics, Toronto, 2020. 1095–1102
10 Ravber M, Smid M, Moravec M, et al. Performance comparison of single-objective evolutionary algorithms implemented in different frameworks. Informatica, 2025, 36: 677–712
11 Tian Y, Cheng R, Zhang X, et al. PlatEMO: a MATLAB platform for evolutionary multi-objective optimization. IEEE Comput Intell Mag, 2017, 12: 73–87
12 Lopez-Ibanez M, Branke J, Paquete L. Reproducibility in evolutionary computation. ACM Trans Evol Learn Optim, 2021, 1: 14:1–14:21
13 Yang L, Zhang Y, Cao J, et al. A many-objective evolutionary algorithm based on reference vector guided selection and two diversity and convergence enhancement strategies. Appl Soft Comput, 2024, 154: 111369
14 Liang J, Zhang Y Y, Chen K, et al. An evolutionary multiobjective method based on dominance and decomposition for feature selection in classification. Sci China Inf Sci, 2024, 67: 120101
15 Derrac J, Garcia S, Molina D, et al. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. Swarm Evol Comput, 2011, 1: 3–18
16 Demsar J. Statistical comparisons of classifiers over multiple data sets. J Mach Learn Res, 2006, 7: 1–30
17 Lian J J. Time-fair benchmarking for metaheuristics: a restart-fair protocol for fixed-time comparisons. arXiv:2509.08986, 2025
18 Molina D, Del Ser J, Poyatos J, et al. The paradox of success in evolutionary and bioinspired optimization: revisiting critical issues, key studies, and methodological pathways. Swarm Evol Comput, 2025, 98: 102063
19 Kononova A V, van Stein N, Mersmann O, et al. Benchmarking that matters: rethinking benchmarking for practical impact. arXiv:2511.12264, 2025
20 Yao S, Liu F, Lin X, et al. Multi-objective evolution of heuristic using large language model. In: Proceedings of the AAAI Conference on Artificial Intelligence, Philadelphia, 2025. 27144–27152
21 Zhou M C, Cui M, Xu D, et al. Evolutionary optimization methods for high-dimensional expensive problems: a survey. IEEE/CAA J Autom Sinica, 2024, 11: 1092–1105
22 Liang J, Lou Y, Yu M, et al. A survey of surrogate-assisted evolutionary algorithms for expensive optimization. J Membr Comput, 2024, 7: 108–127
23 Liu Z, Han F, Ling Q, et al. Constraint-Pareto dominance and diversity enhancement strategy-based evolutionary algorithm for solving constrained multiobjective optimization problems. IEEE Trans Evol Comput, 2025, 29: 2771–2784
24 Yang P, Zhang L M, Liu H F, et al. Reducing idleness in financial cloud services via multi-objective evolutionary reinforcement learning based load balancer. Sci China Inf Sci, 2024, 67: 120102
25 Hao H, Zhang X Q, Zhou A. Enhancing SAEAs with unevaluated solutions: a case study of relation model for expensive optimization. Sci China Inf Sci, 2024, 67: 120103
26 Tian Y, Xiang X, Zhang X, et al. Sampling reference points on the Pareto fronts of benchmark multi-objective optimization problems. In: Proceedings of the IEEE Congress on Evolutionary Computation, Rio de Janeiro, 2018. 1–6
27 Das I, Dennis J E. Normal-boundary intersection: a new method for generating the Pareto surface in nonlinear multicriteria optimization problems. SIAM J Optim, 1998, 8: 631–657
28 Blank J, Deb K. Pymoo: multi-objective optimization in Python. IEEE Access, 2020, 8: 89497–89509
29 Wang W, Tan Z, Wang Y, et al. Enhancing the scalability of large-scale multi-objective evolutionary algorithm through frequency domain search. Swarm Evol Comput, 2026, 106: 102423
30 Deb K, Thiele L, Laumanns M, et al. Scalable multi-objective optimization test problems. In: Proceedings of the IEEE Congress on Evolutionary Computation, Honolulu, 2002. 825–830
31 Cheng R, Jin Y, Olhofer M. Test problems for large-scale multiobjective and many-objective optimization. IEEE Trans Cybern, 2017, 47: 4108–4121
32 Zitzler E, Thiele L. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. IEEE Trans Evol Comput, 1999, 3: 257–271
33 Tian Y, Zheng X, Zhang X, et al. Efficient large-scale multi-objective optimization based on a competitive swarm optimizer. IEEE Trans Cybern, 2020, 50: 3696–3708